# Speech to Speech Translation Systems

## Benchmarking and Implementation

Tanay Bhat (22B3303)
Jay Mehta (22B1281)
Kshitij Vaidya (22B1829)
Vatsal Melwani (22B0396)

IIT Bombay

May 1, 2026

# Outline

# Introduction

# ASR: Overview

The ASR module of a S2S pipeline first converts the input speech into text. It involves several key components:

- **Audio Preprocessing:** Noise reduction and feature extraction using MFCCs, Mel spectrograms etc.
- **Encoder:** Responsible for capturing temporal dependencies and contextual information of the input audio using the extracted features. Typically implemented using RNNs, CNNs, or Transformers.
- **Decoder:** Generates the output text sequence based on the encoder reprensentation. Current state-of-the-art models often use attention mechanisms to focus on relevant parts of the input during decoding.

# ASR Benchmarking: Setup

- Accent and dialect variation
- Background noise handling
- Real-time processing requirements
- Domain-specific vocabulary

# ASR Benchmarking: Results

- Accent and dialect variation
- Background noise handling
- Real-time processing requirements
- Domain-specific vocabulary

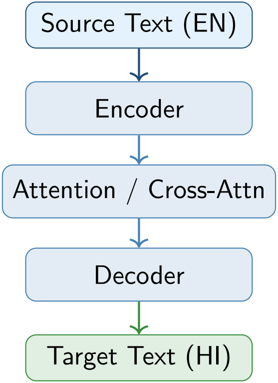
# ASR Benchmarking: Inferences

- Accent and dialect variation
- Background noise handling
- Real-time processing requirements
- Domain-specific vocabulary

# What is Machine Translation?

## Definition

**Machine Translation (MT)** is the automated process of translating text from one natural language to another using computational models, without human intervention.




**Key Paradigms:**

- **Rule-Based MT (RBMT)** — hand-crafted linguistic rules
- **Statistical MT (SMT)** — phrase/word alignment probabilities
- **Neural MT (NMT)** — encoder–decoder deep learning
- **Large Language Models** —

# The English–Hindi Translation Challenge

## Why English → Hindi?

- Hindi has **600M+ speakers** globally
- Significant demand in news, government, e-commerce
- Morphologically rich language with complex agreement
- **Low-resource** relative to European language pairs

## Key Linguistic Challenges

- Script difference: Latin → Devanagari
- Free word order (SOV vs. SVO)
- Agglutinative morphology
- Named entity transliteration
- Code-mixing in source text
- Cultural / idiomatic expressions

## Test Set: IIT Bombay English–Hindi Corpus

A widely used benchmark dataset containing aligned sentence pairs sourced from diverse domains including news, government documents, and

# Systems Under Evaluation

### [NMT] MarianMT (Helsinki-NLP)

- Encoder–decoder Transformer
- Trained on OPUS parallel corpora
- Lightweight, fast inference
- Widely used baseline in research

### [NMT] mBART-50 (Meta AI)

- Multilingual BART pre-training
- Fine-tuned on 50 languages
- Denoising pre-training objective
- Strong cross-lingual transfer

### [NMT] NLLB-600M (Meta AI)

- No Language Left Behind initiative
- 200-language multilingual model

### [API] Google Translate (API)

- Commercial neural MT system
- Massive proprietary training data
- Continuously updated model

# MT: Key Approaches

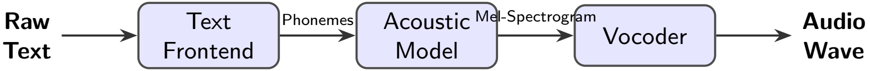
- Neural Machine Translation (NMT)
- Transfer learning and multilingual models
- Low-resource language handling
- Context-aware translation

# What is Text-to-Speech (TTS)?

**Text-to-Speech (TTS)** is the artificial synthesis of human-sounding speech from raw text.

- **Core Objectives:**
  - *Intelligibility:* Clear, understandable pronunciation of words.
  - *Naturalness:* Accurate prosody, intonation, rhythm, and emotion.
- **Evolution of the Technology:**
  - **Concatenative (1990s-2000s):** Splicing together tiny fragments of pre-recorded human speech. Highly intelligible but sounded robotic and rigid.
  - **Parametric (2000s-2010s):** Statistical models (like HMMs) generating audio based on parameters. Smoother, but sounded muffled or "buzzy".
  - **Neural (Present):** Deep learning models predicting high-fidelity audio waves directly from text, achieving near-human naturalness.

# The Standard Neural TTS Pipeline




- **Text Frontend:** Cleans and normalizes text (e.g., expanding "180" to "one hundred eighty") and performs Grapheme-to-Phoneme (G2P) conversion to determine pronunciation.
- **Acoustic Model:** Maps phonemes to acoustic representations, predicting pitch, energy, and duration. *(Examples: Tacotron 2, FastSpeech)*
- **Vocoder:** Reconstructs the actual time-domain audio waveform from the highly compressed Mel-spectrogram. *(Examples: WaveNet, HiFi-GAN)*

# Systems Under Evaluation

### MMS-TTS

- **Architecture:** VITS-based (End-to-End).
- **Primary Strength:** Highly lightweight and efficient; state-of-the-art for low-resource languages.
- **Limitation:** Generally single-speaker per model with fixed prosody.

### Parler-TTS

- **Architecture:** Autoregressive Transformer.
- **Primary Strength:** Granular control over emotion, tone, pacing, and background noise via text descriptions.
- **Limitation:** Higher inference latency due to autoregressive generation.

### XTTS-v2

- **Architecture:** GPT-style + Mel/HiFi-GAN.
- **Primary Strength:** Requires only $\sim$3 seconds of reference audio; supports cross-lingual voice cloning.
- **Limitation:** High resource consumption compared to non-autoregressive models.

# Evaluation Metrics for TTS Models

## Intelligibility & Efficiency

- **Word Error Rate (WER)**
  *What it is:* Transcription accuracy of the audio using an ASR system. Lower is better.
- **Character Error Rate (CER)**
  *What it is:* Similar to WER, but captures finer phonetic/spelling errors. Lower is better.
- **Real-Time Factor (RTF)**
  *What it is:* Processing time divided by generated audio length. Lower is better.

## Prosody & Expressiveness

- **Pitch Variance ($F_0$ Std Dev)**
  *What it is:* Range of intonation.
- **Energy Dynamics (RMS Std)**
  *What it is:* Variations in volume/loudness.
- **Speaking Rate**
  *What it is:* Speed (Syllables/words per sec).
- **Pause Ratio (Onset/s)**
  *What it is:* Frequency of breath/silence pauses.

# TTS: Benchmarking Results

| Model | Language | Latency (ms) | RTF | Throughput (cps) |
|---|---|---|---|---|
| MMS-TTS | English | 2621.382 | 0.439 | 31.711 |
| MMS-TTS | Hindi | 2451.2891 | 0.432 | 27.49 |
| Parler-TTS | English | 68026.311 | 11.264 | 1.281 |
| XTTS-v2 | English | 26182.9 | 3.438 | 3.491 |
| XTTS-v2 | Hindi | 23687.88 | 3.204 | 2.718 |

Table: Benchmarking results for TTS models across Performance Metrics.

# TTS: Benchmarking Results

| Model | Language | WER | CER |
|---|---|---|---|
| MMS-TTS | English | 34.892 | 17.968 |
| MMS-TTS | Hindi | - | - |
| Parler-TTS | English | 27.57 | 19.854 |
| XTTS-v2 | English | 20.465 | 6.825 |
| XTTS-v2 | Hindi | - | - |

Table: Benchmarking results for TTS models across Performance Metrics (Contd.).

**WER/CER for Hindi were turning out to be extremely high due to ASR model's poor performance on Hindi audio, so we have omitted those values to avoid skewing the analysis.*

# TTS: Benchmarking Results

| Model | Language | Pitch (Mean) | Pitch (Std) | Pitch Range |
|---|---|---|---|---|
| MMS-TTS | English | 99.978 | 19.158 | 81.388 |
| MMS-TTS | Hindi | 148.216 | 37.359 | 176.392 |
| Parler-TTS | English | 233.055 | 54.52 | 253.809 |
| XTTS-v2 | English | 141.029 | 29.674 | 136.078 |
| XTTS-v2 | Hindi | 139.625 | 16.915 | 76.358 |

Table: Benchmarking results for TTS models across Prosodic Metrics (Contd.).

# TTS: Benchmarking Results

| Model | Language | Speaking Rate | Energy Dynamics | Pause Ratio |
|---|---|---|---|---|
| MMS-TTS | English | 6.788 | 0.067 | 0.183 |
| MMS-TTS | Hindi | 5.808 | 0.093 | 0.246 |
| Parler-TTS | English | 5.35 | 0.082 | 0.153 |
| XTTS-v2 | English | 4.284 | 0.099 | 0.212 |
| XTTS-v2 | Hindi | 4.268 | 0.1002 | 0.186 |

Table: Benchmarking results for TTS models across Prosodic Metrics.

# TTS: Plots

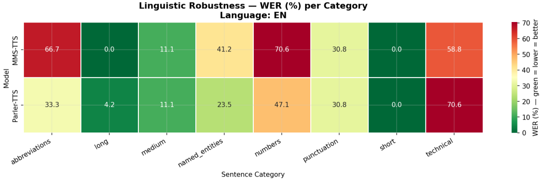



Figure: WER Heatmap for English for MMS-TTS and Parler-TTS for different types of sentences.

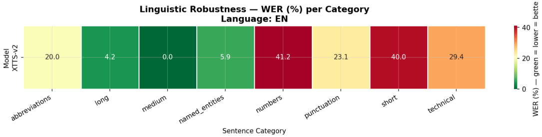



Figure: WER Heatmap for English for XTTS-v2 for different types of sentences.

# End-to-End Pipeline

- ASR $\rightarrow$ MT $\rightarrow$ TTS workflow
- Use case: Real-time speech translation
- Latency and quality trade-offs
- Error propagation considerations

# Pipeline Architecture

- Modular vs. end-to-end approaches
- API integration strategies
- Scalability and deployment
- Performance optimization

# Applications

- International conference interpretation
- Language learning platforms
- Accessibility tools
- Customer service automation

# Conclusion

- Integrated ASR-MT-TTS systems enable powerful applications
- Continued advances in neural architectures
- Challenges remain in low-resource scenarios
- Future directions: multimodal integration, personalization

# Thank You

Thank you for your attention!

Questions?

Contact: your.email@example.com